Modular

# How Mojo supports non-LLVM back-ends (TPU & Trainium)




TPU




Trainium

**Kolya Panchenko**

Modular

LLVM Developers' Meeting · Mojo workshop · October 2026

# TPU has a real vector unit; the rest matches Trainium







| | TPU | Trainium |
|---|---|---|
| Compute | MXU systolic array, VPU with 2-D vregs, scalar unit | Tensor, Vector and Scalar engines |
| Model | load to vreg → compute on vector → store | ops on tiles in SBUF/PSUM, view encoded on the op |
| Registers | 2-D: sublane × lane, 8×128 | no usable SIMD registers |

# TPU has a real vector unit; the rest matches Trainium






| | TPU | Trainium |
|---|---|---|
| Compute | MXU systolic array, VPU with 2-D vregs, scalar unit | Tensor, Vector and Scalar engines |
| Model | load to vreg → compute on vector → store | ops on tiles in SBUF/PSUM, view encoded on the op |
| Registers | 2-D: sublane × lane, 8×128 | no usable SIMD registers |

**Shared constraints.** Tensor-based, static shapes, almost no control flow, no function calls.

# Google’s stack maps onto AWS’s, with one extra hop



# Google’s stack maps onto AWS’s, with one extra hop




| TPU (Google) | Trainium (AWS) | Nature |
|---|---|---|
| Pallas + Mosaic | NKI | open, MLIR |
| libtpu | Neuron Compiler | closed |
| TPU executable | NEFF | opaque binary vs JSON archive |
| PJRT | NRT | C API |

# A TPU vector add, written directly in Mosaic MLIR

THE SIGNATURE: THREE BUFFERS IN VMEM

```
func.func @vector_add(
    %lhs: memref<128xf32, #tpu.memory_space<vmem>>,
    %rhs: memref<128xf32, #tpu.memory_space<vmem>>,
    %dst: memref<128xf32, #tpu.memory_space<vmem>>)
    attributes {tpu.core_type = #tpu.core_type<tc>} {
  …
}
```

# A TPU vector add, written directly in Mosaic MLIR

THE BODY: LOAD, ADD, ON WHOLE VECTORS

```
func.func @vector_add(
    %lhs: memref<128xf32, #tpu.memory_space<vmem>>,
    %rhs: memref<128xf32, #tpu.memory_space<vmem>>,
    %dst: memref<128xf32, #tpu.memory_space<vmem>>)
    attributes {tpu.core_type = #tpu.core_type<tc>} {
  %c0 = arith.constant 0 : index
  %a = vector.load %lhs[%c0] : memref<128xf32, #tpu.memory_space<vmem>>, vector<128xf32>
  %b = vector.load %rhs[%c0] : memref<128xf32, #tpu.memory_space<vmem>>, vector<128xf32>
  %sum = arith.addf %a, %b : vector<128xf32>
  …
}
```

# A TPU vector add, written directly in Mosaic MLIR

THE STORE: BACK INTO VMEM

```
func.func @vector_add(
    %lhs: memref<128xf32, #tpu.memory_space<vmem>>,
    %rhs: memref<128xf32, #tpu.memory_space<vmem>>,
    %dst: memref<128xf32, #tpu.memory_space<vmem>>)
    attributes {tpu.core_type = #tpu.core_type<tc>} {
  %c0 = arith.constant 0 : index
  %a = vector.load %lhs[%c0] : memref<128xf32, #tpu.memory_space<vmem>>, vector<128xf32>
  %b = vector.load %rhs[%c0] : memref<128xf32, #tpu.memory_space<vmem>>, vector<128xf32>
  %sum = arith.addf %a, %b : vector<128xf32>
  tpu.vector_store %dst[%c0], %sum {strides = array<i32>}
      : memref<128xf32, #tpu.memory_space<vmem>>, vector<128xf32>,
  return
}
```

That is the whole kernel: typed memrefs in vmem, vector ops, one store. Anything that can emit this IR can target the TPU.

# Mojo is a general-purpose language: it has no memref

| | Mojo has | Mosaic expects | Gap |
|---|---|---|---|
| Scalar types | Int, Float32, Bool | | |
| Vector types | SIMD[dtype, N], 1-D | | |
| Memory types | Pointer[T, address_space]: an address, nothing else | | |
| Aggregate types | Structs | | |
| Control flow | Calls, unstructured CFG | | |

# Mojo is a general-purpose language: it has no memref

| | Mojo has | Mosaic expects | Gap |
|---|---|---|---|
| Scalar types | Int, Float32, Bool | i32, f32, index (in smem) | Direct mapping |
| Vector types | SIMD[dtype, N], 1-D | vector<8×128xf32>, 2-D tiles | Rank mismatch |
| Memory types | Pointer[T, address_space]: an address, nothing else | memref<128×128xbf16, vmem>: shape, layout, memory space | No equivalent |
| Aggregate types | Structs | None at the kernel ABI | Scalarize (SROA) |
| Control flow | Calls, unstructured CFG | Single function, structured control flow (scf) only | Full inlining |

**Nothing in Mojo is a memref.** Pointers, scalars, SIMD vectors and structs are all a CPU or GPU needs; the kernel we just saw is built from none of them.

# Type question: can a memref be rebuilt from a pointer?

IN MOJO: A POINTER, WITH THE REST AS METADATA

```
struct TPUBuffer[dtype: DType, rows: Int, cols: Int, space: AddressSpace]:
    var ptr: Pointer[Scalar[Self.dtype], MutAnyOrigin, address_space=Self.space]
    # shape, layout and memory space exist only as parameters
```

→ the lowering must find all three pieces again and reassemble

```
memref<128x128xbf16, #tpu.memory_space<vmem>>
```

# Type question: can a memref be rebuilt from a pointer?

IN MOJO: A POINTER, WITH THE REST AS METADATA

```
struct TPUBuffer[dtype: DType, rows: Int, cols: Int, space: AddressSpace]:
    var ptr: Pointer[Scalar[Self.dtype], MutAnyOrigin, address_space=Self.space]
    # shape, layout and memory space exist only as parameters
```

→ the lowering must find all three pieces again and reassemble

```
memref<128x128xbf16, #tpu.memory_space<vmem>>
```

### WHY IT IS MESSY

Every pass between Mojo and Mosaic (inlining, SROA, closures, structs) must keep the pointer and its metadata together. Lose the pairing and the kernel is silently different.

### WE ALREADY DO THIS FOR APPLE GPU

Metal needs address spaces Mojo pointers don't always carry, so we infer them on LLVM IR. A fused concat with three inputs read zeros because an outlined lambda lost the device address space. Each fix is one more pattern.

# Mojo is already syntax sugar for MLIR

WHAT YOU WRITE

```
def vector_add[dtype: DType, N: SIMDLength](
    a: SIMD[dtype, N], b: SIMD[dtype, N]
) → SIMD[dtype, N]:
    return a + b
```

# Mojo is already syntax sugar for MLIR

WHAT YOU WRITE

```
def vector_add[dtype: DType, N: SIMDLength](
    a: SIMD[dtype, N], b: SIMD[dtype, N]
) → SIMD[dtype, N]:
    return a + b
```

WHAT THE STANDARD LIBRARY UNFOLDS TO

```
def vector_add[dtype: __mlir_type.`!kgen.dtype`, N: __mlir_type.index](
    a: __mlir_type[`!kgen.simd<`, N, `, `, dtype, `>`],
    b: __mlir_type[`!kgen.simd<`, N, `, `, dtype, `>`],
) → __mlir_type[`!kgen.simd<`, N, `, `, dtype, `>`]:
    return __mlir_op.`pop.add`(a, b)
```

# Mojo is already syntax sugar for MLIR

WHAT YOU WRITE

```
def vector_add[dtype: DType, N: SIMDLength](
    a: SIMD[dtype, N], b: SIMD[dtype, N]
) → SIMD[dtype, N]:
    return a + b
```

WHAT THE STANDARD LIBRARY UNFOLDS TO

```
def vector_add[dtype: __mlir_type.`!kgen.dtype`, N: __mlir_type.index](
    a: __mlir_type[`!kgen.simd<`, N, `, `, dtype, `>`],
    b: __mlir_type[`!kgen.simd<`, N, `, `, dtype, `>`],
) → __mlir_type[`!kgen.simd<`, N, `, `, dtype, `>`]:
    return __mlir_op.`pop.add`(a, b)
```

So if Mojo can spell our own dialects, it can spell the vendor's. Kernels write Mosaic (or trn) directly.

# First milestone: vector add compiles

```
comptime M =
    __mlir_type.`memref<128xf32, #tpu.memory_space<vmem>>`
comptime V = __mlir_type.`vector<128xf32>`

def vector_add(
    lhs: M, rhs: M, dst: M
):
    var c0 = __mlir_op.`arith.constant`[
        _type=__mlir_type.index, value=__mlir_attr.`0 : index`
    ]()
    var a = __mlir_op.`vector.load`[_type=V](lhs, c0)
    var b = __mlir_op.`vector.load`[_type=V](rhs, c0)
    var tmp = __mlir_op.`arith.addf`[_type=V](a, b)
    __mlir_op.`tpu.vector_store`[_type=None, strides=__mlir_attr.`array<i32>`](tmp, dst, c0)
```

# First milestone: vector add compiles

```
comptime M = __mlir_type.`memref<128xf32, #tpu.memory_space<vmem>>`
comptime V = __mlir_type.`vector<128xf32>`

def vector_add(lhs: M, rhs: M, dst: M):
    var c0 = __mlir_op.`arith.constant`[_type=__mlir_type.index, value=__mlir_attr.`0 : index`]()
    var a = __mlir_op.`vector.load`[_type=V](lhs, c0)
    var b = __mlir_op.`vector.load`[_type=V](rhs, c0)
    var tmp = __mlir_op.`arith.addf`[_type=V](a, b)
    __mlir_op.`tpu.vector_store`[_type=None](tmp, dst, c0)
```

**MOJO'S ELABORATOR NOW GENERATES: KGEN · MOSAIC**

```
!M = memref<128xf32, #tpu.memory_space<vmem>>
kgen.func export @"vector_add"(%lhs: !M, %rhs: !M, %dst: !M) {
  %c0 = arith.constant 0 : index
  %a = vector.load %lhs[%c0] : !M, vector<128xf32>
  %b = vector.load %rhs[%c0] : !M, vector<128xf32>
  %tmp = arith.addf %a, %b : vector<128xf32>
  tpu.vector_store %dst[%c0], %tmp : !M, vector<128xf32>,
  hlcf.return
}
```

# To reach Mosaic, only two ops need lowering

AFTER ELABORATION

```
!M = memref<128xf32, #tpu.memory_space<vmem>>
kgen.func export @"vector_add"(%lhs: !M, %rhs: !M, %dst: !M) {
  %c0 = arith.constant 0 : index
  %a = vector.load %lhs[%c0] : !M, vector<128xf32>
  %b = vector.load %rhs[%c0] : !M, vector<128xf32>
  %tmp = arith.addf %a, %b : vector<128xf32>
  tpu.vector_store %dst[%c0], %tmp : !M, vector<128xf32>,
  hlcf.return
}
```

Everything inside the function is already Mosaic. Only kgen.func and hlcf.return are Mojo's own ops.

# To reach Mosaic, only two ops need lowering

AFTER LOWER-TO-MOSAIC

```
!M = memref<128xf32, #tpu.memory_space<vmem>>
func.func @vector_add(%lhs: !M, %rhs: !M, %dst: !M) attributes {tpu.core_type = #tpu.core_type<tc>} {
  %c0 = arith.constant 0 : index
  %a = vector.load %lhs[%c0] : !M, vector<128xf32>
  %b = vector.load %rhs[%c0] : !M, vector<128xf32>
  %tmp = arith.addf %a, %b : vector<128xf32>
  tpu.vector_store %dst[%c0], %tmp : !M, vector<128xf32>,
  return
}
```

kgen.func → func.func with the TPU core type; hlcf.return → return. That is the whole lowering for this kernel.

# Mojo compiles; the driver runs

COMPILE TIME

## Mojo compiler

Parses and elaborates the kernel

Lowers it to the target dialect

Calls the vendor compiler (libtpu)

Embeds the executable in the host binary



`executable`

RUN TIME

## Driver

Loads the executable on the device

Allocates device buffers

Copies host ↔ device

Launches the kernel and waits for it

**Two separate jobs.** On GPU the driver is CUDA or HIP behind DeviceContext; on TPU it speaks PJRT, through libtpu.

# No driver yet? Call PjRT through FFI

```
var info = compile_info_tpu[vector_add]()   # Mosaic → libtpu AOT
var executable = info.asm.unsafe_ptr()  # embedded TPU executable
var size = info.asm.byte_length()

# libtpu exports one symbol; everything else is a slot in its table
var api = external_call["GetPjrtApi", OpaquePointer[MutAnyOrigin]]()
# pjrt_call(api, SLOT, args...): call slot SLOT of that table
var client = pjrt_call(api, CLIENT_CREATE)

var exe = pjrt_call(api, DESERIALIZE_AND_LOAD, client, executable, size)
var a = pjrt_call(api, BUFFER_FROM_HOST, client, lhs.unsafe_ptr(), n)
var b = pjrt_call(api, BUFFER_FROM_HOST, client, rhs.unsafe_ptr(), n)
var res = pjrt_call(api, EXECUTE, exe, a, b)
pjrt_call(api, TO_HOST_BUFFER, res, out.unsafe_ptr(), n)
```

# Now make vector add parametric

```
comptime M =
    __mlir_type.`memref<128xf32, #tpu.memory_space<vmem>>`
comptime V = __mlir_type.`vector<128xf32>`

def vector_add(
    lhs: M, rhs: M, dst: M
):
    var c0 = __mlir_op.`arith.constant`[
        _type=__mlir_type.index, value=__mlir_attr.`0 : index`
    ]()
    var a = __mlir_op.`vector.load`[_type=V](lhs, c0)
    var b = __mlir_op.`vector.load`[_type=V](rhs, c0)
    var tmp = __mlir_op.`arith.addf`[_type=V](a, b)
    __mlir_op.`tpu.vector_store`[_type=None, strides=__mlir_attr.`array<i32>`](tmp, dst, c0)
```

# Now make vector add parametric

```
comptime M[dtype: DType, N: Int] =
    __mlir_type[`memref<`, N, `x`, dtype, `, #tpu.memory_space<vmem>>`]
comptime V[dtype: DType, N: Int] = __mlir_type[`vector<`, N, `x`, dtype, `>`]

def vector_add[dtype: DType, N: Int](
    lhs: M[dtype, N], rhs: M[dtype, N], dst: M[dtype, N]
):
    var c0 = __mlir_op.`arith.constant`[
        _type=__mlir_type.index, value=__mlir_attr.`0 : index`
    ]()
    var a = __mlir_op.`vector.load`[_type=V[dtype, N]](lhs, c0)
    var b = __mlir_op.`vector.load`[_type=V[dtype, N]](rhs, c0)
    var tmp = __mlir_op.`arith.addf`[_type=V[dtype, N]](a, b)
    __mlir_op.`tpu.vector_store`[_type=None, strides=__mlir_attr.`array<i32>`](tmp, dst, c0)
```

# Now make vector add parametric

```
comptime M[dtype: DType, N: Int] =
    __mlir_type[`memref<`, N, `x`, dtype, `, #tpu.memory_space<vmem>>`]
comptime V[dtype: DType, N: Int] = __mlir_type[`vector<`, N, `x`, dtype, `>`]

def vector_add[dtype: DType, N: Int](
    lhs: M[dtype, N], rhs: M[dtype, N], dst: M[dtype, N]
):
    var c0 = __mlir_op.`arith.constant`[
        _type=__mlir_type.index, value=__mlir_attr.`0 : index`
    ]()
    var a = __mlir_op.`vector.load`[_type=V[dtype, N]](lhs, c0)
    var b = __mlir_op.`vector.load`[_type=V[dtype, N]](rhs, c0)
    var tmp = __mlir_op.`arith.addf`[_type=V[dtype, N]](a, b)
    __mlir_op.`tpu.vector_store`[_type=None, strides=__mlir_attr.`array<i32>`](tmp, dst, c0)
```

**memref and vector cannot be parametric on shape or element type.** Mojo builds MLIR types while parsing, when N and dtype are still symbolic, but the memref and vector parsers need concrete values: rejected before the kernel is ever instantiated.

# Deferred types and attributes are MLIR text, parsed at elaboration

```
def get_llvm_array[n: Int]() → __mlir_deferred_type[
    `!llvm.array<`, +n.__mlir_index__(), ` x f32>`
]:
    ...

comptime llvm_array4 = get_llvm_array[4]()    # becomes !llvm.array<4 x f32> at elaboration

def _overflow_kind[signed: Bool]() → StaticString:
    return "nsw" if signed else "nuw"

comptime props[signed: Bool] = __mlir_deferred_attr[
    `{overflowFlags = #llvm.overflow<`, +_get_kgen_string[_overflow_kind[signed]()](),
    `>}`]

var sum = __mlir_op.`llvm.add`[_type=..., _properties=props[False]](a, b)  # nuw
```

# Deferred types and attributes make memref parametric

BUILT AT ELABORATION TIME

```
comptime mnemonic[dtype: DType] = _get_kgen_string[_dtype_mnemonic[dtype]()]()
comptime M[dtype: DType, N: Int] = __mlir_deferred_type[
    `memref<`, +N.__mlir_index__(), `x`, +mnemonic[dtype], `, #tpu.memory_space<vmem>>`
]
comptime V[dtype: DType, N: Int] = __mlir_deferred_type[
    `vector<`, +N.__mlir_index__(), `x`, +mnemonic[dtype], `>`
]
```

# Parametric vector add, with deferred types

```
comptime mnemonic[dtype: DType] = _get_kgen_string[_dtype_mnemonic[dtype]()]()
comptime M[dtype: DType, N: Int] = __mlir_deferred_type[
    `memref<`, +N.__mlir_index__(), `x`, +mnemonic[dtype], `, #tpu.memory_space<vmem>>`
]
comptime V[dtype: DType, N: Int] = __mlir_deferred_type[
    `vector<`, +N.__mlir_index__(), `x`, +mnemonic[dtype], `>`
]

def vector_add[dtype: DType, N: Int](
    lhs: M[dtype, N], rhs: M[dtype, N], dst: M[dtype, N]
):
    var c0 = __mlir_op.`arith.constant`[
        _type=__mlir_type.index, value=__mlir_attr.`0 : index`
    ]()
    var a = __mlir_op.`vector.load`[_type=V[dtype, N]](lhs, c0)
    var b = __mlir_op.`vector.load`[_type=V[dtype, N]](rhs, c0)
    var tmp = __mlir_op.`arith.addf`[_type=V[dtype, N]](a, b)
    __mlir_op.`tpu.vector_store`[_type=None, strides=__mlir_attr.`array<i32>`](tmp, dst, c0)
```

# Structs cannot cross the kernel boundary

```
struct MemRef[N: Int]:
    comptime T = __mlir_deferred_type[
        `memref<`, +N.__mlir_index__(), `xf32, vmem>`]
    var memref: Self.T

def kernel[N: Int](x: MemRef[N]): ...
```

→ elaborates to

```
kgen.func @kernel(
    %arg0: !kgen.struct<memref<128xf32, #tpu.memory_space<vmem>>>>
) {
  %t = kgen.struct.extract %arg0[0]
  // use of %t
```

# Structs cannot cross the kernel boundary

```
struct MemRef[N: Int]:
    comptime T = __mlir_deferred_type[
        `memref<`, +N.__mlir_index__(), `xf32, vmem>`]
    var memref: Self.T

def kernel[N: Int](x: MemRef[N]): ...
```

→ elaborates to

```
kgen.func @kernel(
    %arg0: !kgen.struct<memref<Nxf32, #tpu.memory_space<vmem>>>>
) {
  %t = kgen.struct.extract %arg0[0]
  // use of %t
```

→ DecomposeFunctionArguments

```
kgen.func @kernel(%arg0: memref<Nxf32, #tpu.memory_space<vmem >>>)
{
  // use of %arg0
```

# Which arguments are outputs?

**DEFAULT**

The last argument is the single output.

```
def kernel(a: MemRef[..., mut=False], b: MemRef[..., mut=False],
           output: MemRef[..., mut=True]): ...
```

# Which arguments are outputs?

**DEFAULT**

The last argument is the single output.

```
def kernel(a: MemRef[..., mut=False], b: MemRef[..., mut=False],
           output: MemRef[..., mut=True]): ...
```

**BUFFER DONATION**

Like JAX/Pallas input_output_aliases.

```
@__llvm_metadata(
    `tpu.input_output_alias`=StaticTuple[Int32, 2](2, 2)
)
def kernel(a: MemRef[..., mut=False], b: MemRef[..., mut=False],
           output: MemRef[..., mut=True]): ...
```

# Which arguments are outputs?

**DEFAULT**

The last argument is the single output.

```
def kernel(a: MemRef[..., mut=False], b: MemRef[..., mut=False],
           output: MemRef[..., mut=True]): ...
```

**BUFFER DONATION**

Like JAX/Pallas input_output_aliases.

```
@__llvm_metadata(
    `tpu.input_output_alias`=StaticTuple[Int32, 2](2, 2)
)
def kernel(a: MemRef[..., mut=False], b: MemRef[..., mut=False],
           output: MemRef[..., mut=True]): ...
```

**PROBLEM** Users should not annotate every kernel, especially a high-level kernel that runs on several targets.

**SOLUTION: MEMREF CONFORMS TO MUTABILITYKNOWN**

```
struct MemRef[dtype: DType, dims: ..., mut: Bool](MutabilityKnown): ...

def kernel(a: MemRef[..., mut=False], b: MemRef[..., mut=False], output: MemRef[..., mut=True]): ...
# compile time: the TPU hook sees output is mutable and sets tpu.input_output_alias
```

Subject to change: this may move out of the compiler, and then no hook will be needed.

# Closures, without calls or structs

```
def add_kernel_capture_all[OpType: ImplicitlyCopyable & def() → None](op: OpType):
    op()

def main():
    var a = MemRef[DType.float32, 128](...)
    var b = MemRef[DType.float32, 128](...)
    var output = MemRef[DType.float32, 128, mut=True](...)

    @inline(.always)
    def do_add_all() {imm a, imm b, var output} → None:
        ...

    var compiled = ctx.compile_function_unchecked[
        add_kernel_capture_all[type_of(do_add_all)]
    ]()

    ctx.enqueue_function(compiled)
```

# Closures, without calls or structs

```
def add_kernel_capture_all[OpType: ImplicitlyCopyable & def() → None](op: OpType):
    op()

def main():
    var a = MemRef[DType.float32, 128](...)
    var b = MemRef[DType.float32, 128](...)
    var output = MemRef[DType.float32, 128, mut=True](...)

    @inline(.always)
    def do_add_all() {imm a, imm b, var output} → None:
        ...

    var compiled = ctx.compile_function_unchecked[
        add_kernel_capture_all[type_of(do_add_all)]
    ]()

    ctx.enqueue_function(compiled)
```

**CAPTURES BECOME ARGUMENTS**

Captured tensors turn into explicit vmem memref arguments; the host packs them in the same order.

**OUTPUTS GO LAST**

Written captures move to the end, where the wrapper expects outputs.

# Verify target ops when the target is ready

```
def kernel(lhs: MemRef[DType.float32, 8, 128]):
    var v = __mlir_op.`tpu.shuffled_load`[
        _type=type_of(lhs._storage).VectorType,
        sublane_mask=__mlir_attr.`array<i1: ...>`,
        sublane_offsets=__mlir_attr.`array<i32: ...>`,
    ](lhs._storage.memref, c0, c0)
```

↓ elaborates to

```
kgen.func @kernel(%arg0: !kgen.struct<(memref<8x128xf32, vmem>)>) {
  %0 = kgen.struct.extract %arg0[0]
  %1 = tpu.shuffled_load %0[%c0, %c0]
      {sublane_mask = array<i1: ...>, sublane_offsets = array<i32: ...>}
      : memref<8x128xf32, vmem>, vector<8x128xf32>
```

# Inline MLIR in the dialect's own syntax

THE OP IN MOSAIC

```
%res = tpu.matmul %lhs, %rhs, %acc
    {transpose_lhs_hint = false, precision = #tpu.contract_precision<bf16>}
    : vector<8x128xbf16>, vector<128x128xbf16>, vector<8x128xf32> → vector<8x128xf32>
```

# Inline MLIR in the dialect's own syntax

THE OP IN MOSAIC

```
%res = tpu.matmul %lhs, %rhs, %acc
    {transpose_lhs_hint = false, precision = #tpu.contract_precision<bf16>}
    : vector<8x128xbf16>, vector<128x128xbf16>, vector<8x128xf32> → vector<8x128xf32>
```

FOR A LONG TIME: MOJO'S GENERIC FORM

```
__mlir_op.`tpu.matmul`[
    transpose_lhs_hint=__mlir_attr.false,
    precision=__mlir_deferred_attr[
        `#tpu.contract_precision<`, +_get_kgen_string[prec](), `>`
    ],  # prec: a parameter, e.g. "bf16"
    _type=__mlir_type.`vector<8x128xf32>`,
](lhs, rhs, acc)
```

# Inline MLIR in the dialect's own syntax

THE OP IN MOSAIC

```
%res = tpu.matmul %lhs, %rhs, %acc
    {transpose_lhs_hint = false, precision = #tpu.contract_precision<bf16>}
    : vector<8x128xbf16>, vector<128x128xbf16>, vector<8x128xf32> → vector<8x128xf32>
```

NOW: THE ASSEMBLY FORMAT, THE SAME TEXT AS ABOVE

```
comptime p = _get_kgen_string[prec]()  # prec: a parameter, e.g. "bf16"
__mlir_op[
    `tpu.matmul %{lhs}, %{rhs}, %{acc}`,
    ` {transpose_lhs_hint = false,`,
    ` precision = #tpu.contract_precision<%{p}>}`,
    ` : vector<8x128xbf16>, vector<128x128xbf16>, vector<8x128xf32>`,
    ` → vector<8x128xf32>`,
    _type=__mlir_type.`vector<8x128xf32>`,
]
```

# Bringing up new hardware with Mojo

| | | |
|---|---|---|
| 01 | **Study the hardware** | Specs and specifics. |
| 02 | **Study SW stack and IR** | What programs it today |
| 03 | **Map Mojo types** | To the types the target expects |
| 04 | **Map Mojo ops** | To the target's ops, or write them directly with __mlir_op |
| 05 | **Non-parametric "vector add"** | One shape, one element type, inline MLIR |
| 06 | **Add simple lowerings** | Only what it needs |
| 07 | **Run it** | Through the driver, or C FFI if there is no driver yet |
| 08 | **Make "vector add" parametric** | Deferred types and attributes |
| 09 | **Build abstractions around MLIR** | And work around the target compiler's limitations, in your priority order |

Modular

# Thank you

github.com/modular/modular



# In Mosaic, compute is on vectors and the view is just the load

MEMORY SPACES: HBM · VMEM (SCRATCHPAD) · SMEM (SCALARS)

```
%lhs = vector.load %stat[%c0, %c4]
    : memref<128x128xbf16, #tpu.memory_space<vmem>>, vector<8x128xbf16>
%rhs = vector.load %mov[%c0, %c0]
    : memref<128x512xbf16, #tpu.memory_space<vmem>>, vector<128x128xbf16>
%acc = vector.load %dst1[%c3, %c0]
    : memref<128x512xf32, #tpu.memory_space<vmem>>, vector<8x128xf32>
%0 = tpu.matmul %lhs, %rhs, %acc
    {transpose_lhs_hint = false, precision = #tpu.contract_precision<bf16>}
    : vector<8x128xbf16>, vector<128x128xbf16>, vector<8x128xf32> → vector<8x128xf32>
tpu.vector_store %dst1[%c3, %c0], %0 {strides = array<i32>}
    : memref<128x512xf32, #tpu.memory_space<vmem>>, vector<8x128xf32>,
```

# In Mosaic, compute is on vectors and the view is just the load

MEMORY SPACES: HBM · VMEM (SCRATCHPAD) · SMEM (SCALARS)

```
%lhs = vector.load %stat[%c0, %c4]
    : memref<128x128xbf16, #tpu.memory_space<vmem>>, vector<8x128xbf16>
%rhs = vector.load %mov[%c0, %c0]
    : memref<128x512xbf16, #tpu.memory_space<vmem>>, vector<128x128xbf16>
%acc = vector.load %dst1[%c3, %c0]
    : memref<128x512xf32, #tpu.memory_space<vmem>>, vector<8x128xf32>
%0 = tpu.matmul %lhs, %rhs, %acc
    {transpose_lhs_hint = false, precision = #tpu.contract_precision<bf16>}
    : vector<8x128xbf16>, vector<128x128xbf16>, vector<8x128xf32> → vector<8x128xf32>
tpu.vector_store %dst1[%c3, %c0], %0 {strides = array<i32>}
    : memref<128x512xf32, #tpu.memory_space<vmem>>, vector<8x128xf32>,
```

Unlike trn, transpose and precision are matmul attributes, and the matmul never sees an affine map.

# Deferred types and attributes are MLIR text, parsed at elaboration

```
def get_llvm_array[n: Int]() → __mlir_deferred_type[
    `!llvm.array<`, +n.__mlir_index__(), ` x f32>`
]:
    ...
_ = get_llvm_array[4]()    # becomes !llvm.array<4 x f32> at elaboration
comptime props = __mlir_deferred_attr[
    `{overflowFlags = #llvm.overflow<`, +_get_kgen_string[kind](), `>}`
]
```

01 Backtick pieces are MLIR syntax; a +expr piece splices a parameter's value as text.

02 Nothing is built at parse time; ops that use the type or attribute wait too.

03 At elaboration, once parameters are concrete, MLIR's own parser builds the real type or attribute.

Bad text fails at instantiation: invalid MLIR type in deferred_type: !llvm.array<4 bogus f32>

# Deferred types and attributes are MLIR text, parsed at elaboration

```
def get_llvm_array[n: Int]() → __mlir_deferred_type[
    `!llvm.array<`, +n.__mlir_index__(), ` x f32>`
]:
    ...
_ = get_llvm_array[4]()    # becomes !llvm.array<4 x f32> at elaboration
comptime props = __mlir_deferred_attr[
    `{overflowFlags = #llvm.overflow<`, +_get_kgen_string[kind](), `>}`
]
```

01 Backtick pieces are MLIR syntax; a +expr piece splices a parameter's value as text.

02 Nothing is built at parse time; ops that use the type or attribute wait too.

03 At elaboration, once parameters are concrete, MLIR's own parser builds the real type or attribute.

# Same bring-up milestones as for Trainium

| | | |
|---|---|---|
| 01 | Project starts | driver + compiler |
| 02 | Vector add compiles | |
| 03 | Vector add runs | + standard library |

# Same bring-up milestones as for Trainium

| | | |
|---|---|---|
| 01 | Project starts | driver + compiler |
| 02 | Vector add compiles | |
| 03 | Vector add runs | + standard library |
| 04 | Parameters work | + kernels |
| 05 | Abstractions work | |

# Same bring-up milestones as for Trainium

| | | |
|---|---|---|
| 01 | Project starts | driver + compiler |
| 02 | Vector add compiles | |
| 03 | Vector add runs | + standard library |
| 04 | Parameters work | + kernels |
| 05 | Abstractions work | |
| 06 | Kernels are written | + graph compiler |
| 07 | A small model runs | + serving |

# Two independent choices: the type and the ops

| TYPE: WHAT A KERNEL SEES | OPS: WHAT A KERNEL WRITES — UNIFIED DIALECT · SA | VENDOR DIALECT · MOSAIC, TRN |
|---|---|---|
| **Pointer + metadata**<br>memref rebuilt by the compiler | **Two bridges to build**<br>Infer the memref from pointers *and* design a dialect for every systolic array. | **Fragile inference**<br>Vendor ops, but every pass must keep pointer and metadata together. See Apple GPU. |
| **Real memref**<br>built with deferred types | **One more dialect**<br>The type is right, but the unified ops hide what makes each target fast. See xkcd 927. | **What Mojo does**<br>memref through deferred types, Mosaic or trn ops written directly. Nothing new to maintain. |

# What Mojo needs from XLA, and where it plugs in

From openxla/xla, at a pinned commit: the tpu dialect (TableGen, headers) and pjrt_c_api.h

01
**Elaborator**

02
**KGEN / POP**

# What Mojo needs from XLA, and where it plugs in

From openxla/xla, at a pinned commit: the tpu dialect (TableGen, headers) and pjrt_c_api.h

01
**Elaborator**

02
**KGEN / POP**

03
**lower-to-mosaic**

04
**Mosaic**

# Mojo SIMD is 1-D; TPU vector registers are 2-D

POP, FROM MOJO SIMD

```
%v = pop.load %p
  : !kgen.pointer<simd<128, f32>>
pop.store %v, %q
  : !kgen.pointer<simd<128, f32>>
```

# Mojo SIMD is 1-D; TPU vector registers are 2-D

POP, FROM MOJO SIMD

```
%v = pop.load %p
  : !kgen.pointer<simd<128, f32>>
pop.store %v, %q
  : !kgen.pointer<simd<128, f32>>
```



MOSAIC

```
%0 = vector.shape_cast %v
  : vector<128xf32> to vector<1x128xf32>
tpu.vector_store %dst[%c0, %c0], %0
  {strides = array<i32>}
  : memref<1x128xf32, vmem>,
    vector<1x128xf32>,
```

# Mojo SIMD is 1-D; TPU vector registers are 2-D

POP, FROM MOJO SIMD

```
%v = pop.load %p
  : !kgen.pointer<simd<128, f32>>
pop.store %v, %q
  : !kgen.pointer<simd<128, f32>>
```

→

MOSAIC

```
%0 = vector.shape_cast %v
  : vector<128xf32> to vector<1x128xf32>
tpu.vector_store %dst[%c0, %c0], %0
  {strides = array<i32>}
  : memref<1x128xf32, vmem>,
    vector<1x128xf32>,
```

**The opposite of Trainium.** There, SIMD was a dead end because the hardware works on memory. On TPU, SIMD lowers to vector; only the rank is missing.

# Op question: one dialect for all systolic arrays?

| | TPU · Mosaic | Trainium · trn | Unified · sa (hypothetical) |
|---|---|---|---|
| Matmul | tpu.matmul %lhs, %rhs, %acc<br>{transpose_lhs_hint = false}<br>vregs in, vregs out | trn.matmul(dst=%d[3+2*d0, d1],<br>stationary=%s[...],<br>moving=%m[...],<br>is_transpose=false) psum, sbuf | |
| Element wise add | arith.addf %a, %b<br>: vector<8×128xf32> | trn.tensor_tensor_arith<br>{op = add, engine = vector,<br>lhs_ap, rhs_ap, dst_ap} | |
| HBM to on-chip | tpu.enqueue_dma %src, %dst, %sem<br>tpu.wait_dma %sem, ...<br>hbm → vmem | trn.dma_copy<br>{src_ap, dst_ap, dge_mode, ...}<br>shared_hbm → sbuf | |

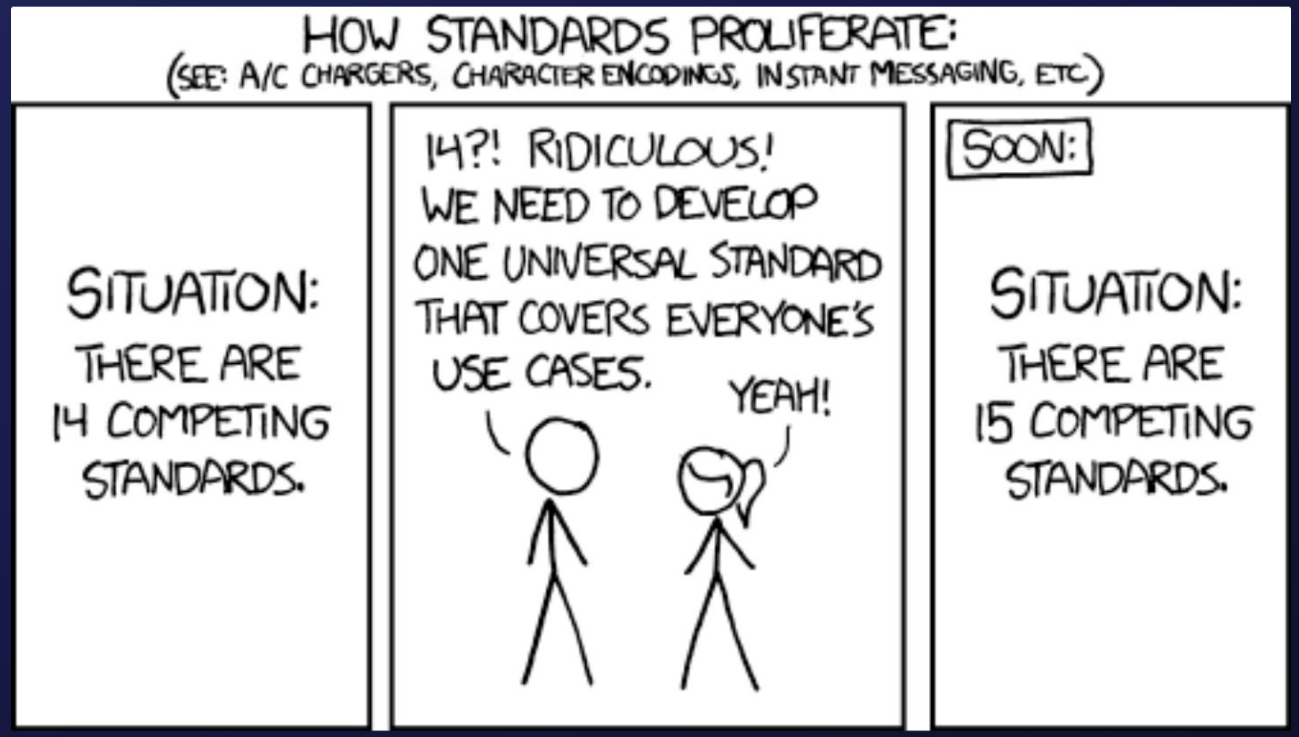
**The op set unifies; what makes each target fast does not.** Registers vs. sbuf and psum, index views vs. affine maps, semaphores vs. DMA modes: each becomes a one-target attribute or lost performance.

# Op question: one dialect for all systolic arrays?

| | TPU · Mosaic | Trainium · trn | Unified · sa (hypothetical) |
|---|---|---|---|
| Matmul | tpu.matmul %lhs, %rhs, %acc<br>{transpose_lhs_hint = false}<br>vregs in, vregs out | trn.matmul(dst=%d[3+2*d0, d1],<br>stationary=%s[...],<br>moving=%m[...],<br>is_transpose=false) psum, sbuf | sa.matmul %acc[v],<br>%lhs[v], %rhs[v]<br>{transpose = false} |
| Element wise add | arith.addf %a, %b<br>: vector<8×128xf32> | trn.tensor_tensor_arith<br>{op = add, engine = vector,<br>lhs_ap, rhs_ap, dst_ap} | sa.elementwise add<br>%dst[v], %a[v], %b[v] |
| HBM to on-chip | tpu.enqueue_dma %src, %dst,<br>%sem<br>tpu.wait_dma %sem, ...<br>hbm → vmem | trn.dma_copy<br>{src_ap, dst_ap, dge_mode, ...}<br>shared_hbm → sbuf | sa.copy %dst[v], %src[v]<br>hbm → on-chip |

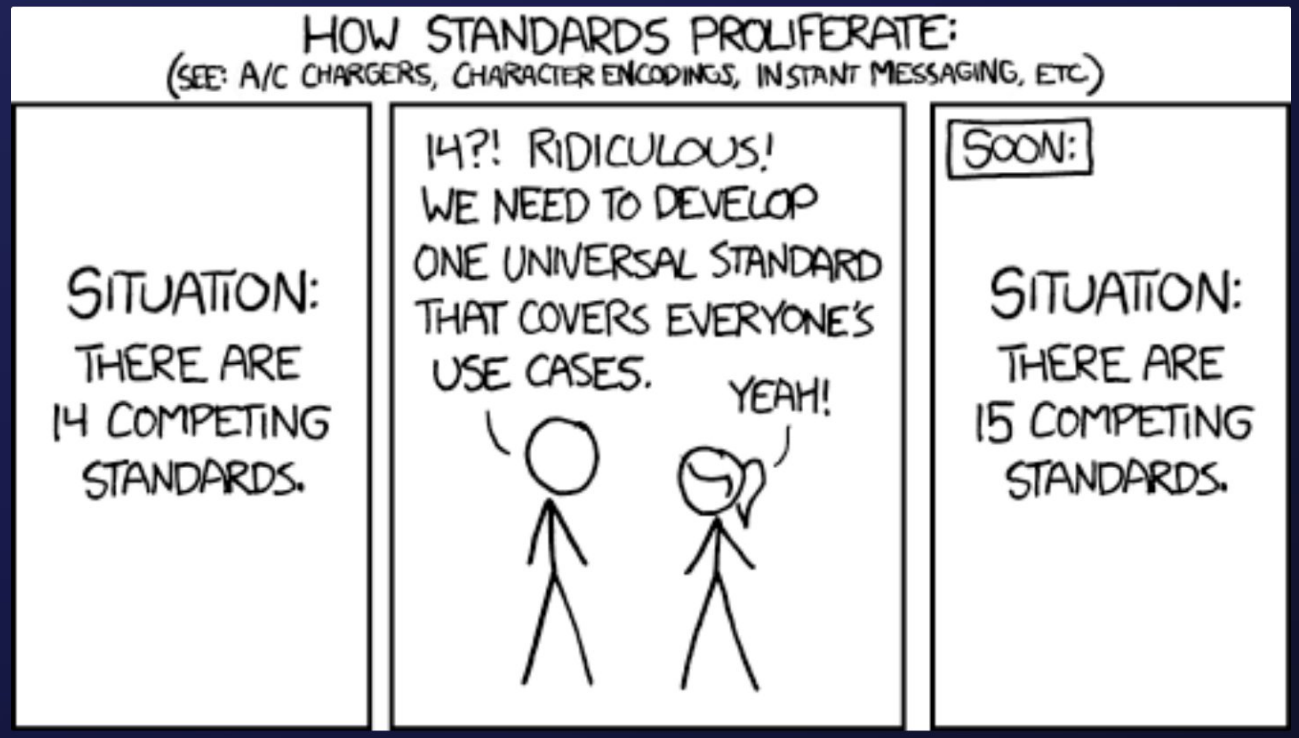
**The op set unifies; what makes each target fast does not.** Registers vs. sbuf and psum, index views vs. affine maps, semaphores vs. DMA modes: each becomes a one-target attribute or lost performance.

# How a unified dialect usually ends




xkcd.com/927, CC BY-NC 2.5

# How a unified dialect usually ends




xkcd.com/927, CC BY-NC 2.5

**WHY IT IS TEMPTING**

One abstraction for TPU- and Trainium-like hardware.

**WHY NOT NOW**

Another dialect to maintain, for a few fast-moving targets.